VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Kiruthika Ramesh (1WA24CS149)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Kiruthika Ramesh **(1WA24CS149)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Pallavi C.V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

LAB PROGRAM 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

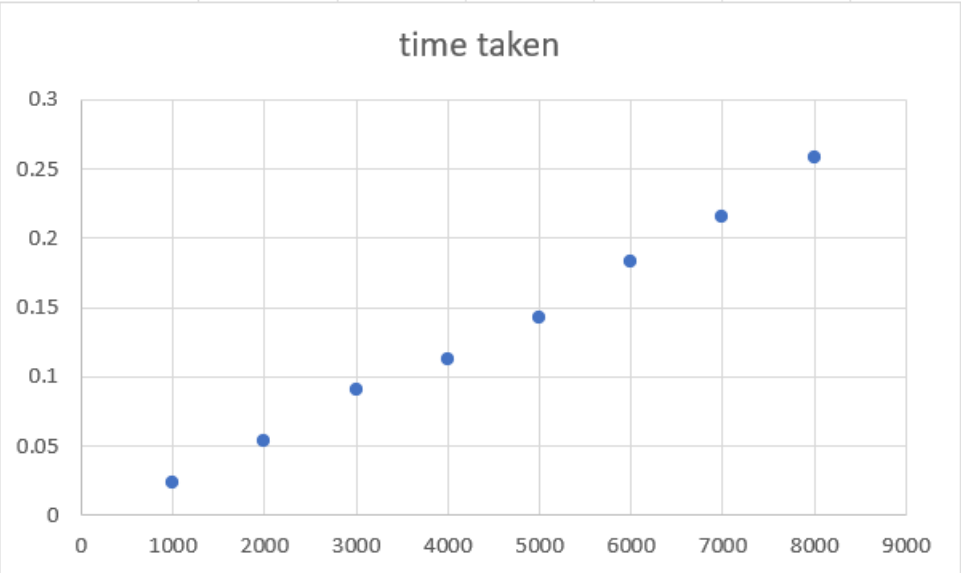
SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int a[20000][20000], v[20000], s[20000], top=-1, n;

void dfs(int i){
   v[i]=1;
   for(int j=0;j<n;j++)
      if(a[i][j] && !v[j]) dfs(j);
   s[++top]=i;
}

int main(){
   int i,j;
   printf("n: "); scanf("%d",&n);

   srand(time(0));

   for(i=0;i<n;i++){
      for(j=0;j<n;j++){
         a[i][j]=(i<j)?rand()%2000:0;
      }
   }

   clock_t t=clock();

   for(i=0;i<n;i++)
      if(!v[i]) dfs(i);

   printf("Topo: ");
   for(i=top;i>=0;i--) printf("%d ",s[i]);

   printf("\nTime: %f", (double)(clock()-t)/CLOCKS_PER_SEC);
}
```

OUTPUT

```
n: 50
Topo: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
Time: 0.013000
Process returned 0 (0x0)   execution time : 9.016 s
Press any key to continue.
```

no. of elements	time taken
1000	0.023
2000	0.054
3000	0.09
4000	0.113
5000	0.143
6000	0.183
7000	0.215
8000	0.258



LAB PROGRAM 2:

Implement Johnson Trotter algorithm to generate permutations.

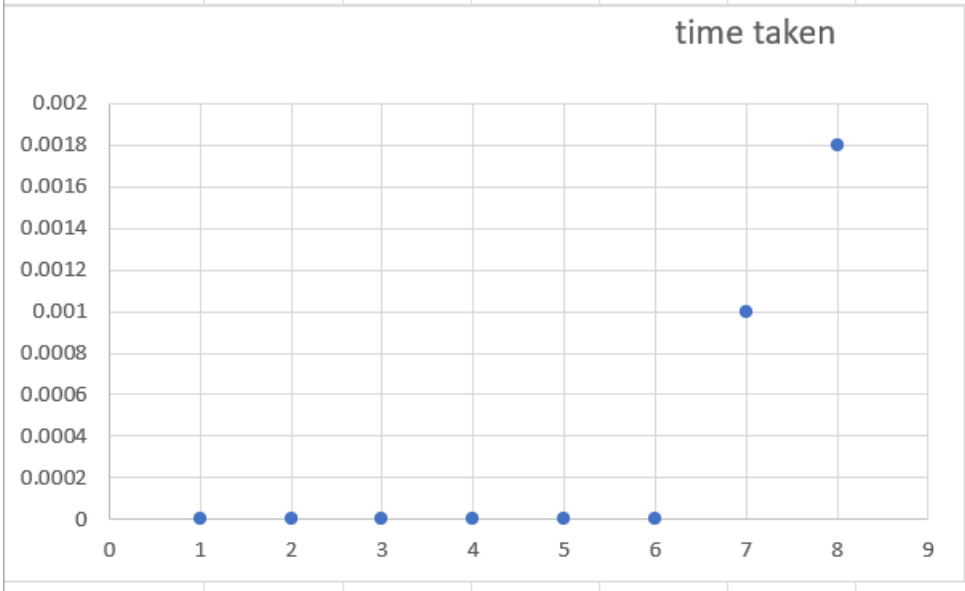
SOURCE CODE

```
#include<stdio.h>
#include<time.h>

int main(){
   int n,i,a[2000],d[2000];
   scanf("%d",&n);

   for(i=0;i<n;i++) a[i]=i+1,d[i]=-1;

   clock_t t=clock();

   while(1){
      int m=0,p=-1;

      for(i=0;i<n;i++){
         int j=i+d[i];
         if(j>=0&&j<n&&a[i]>a[j]&&a[i]>m) m=a[i],p=i;
      }

      if(p==-1) break;

      int j=p+d[p],x=a[p]; a[p]=a[j]; a[j]=x;
      x=d[p]; d[p]=d[j]; d[j]=x;

      for(i=0;i<n;i++) if(a[i]>m) d[i]*=-1;
   }

   printf("Time: %f",(double)(clock()-t)/CLOCKS_PER_SEC);
}
```

OUTPUT

```
1 2 3 4 5 6
Time: 0.000000
Process returned 0 (0x0)   execution time : 11.540 s
Press any key to continue.
```

no. of elements	time taken
1	0
2	0
3	0
4	0
5	0
6	0
7	0.001
8	0.0018
9	0.021



LAB PROGRAM 3:

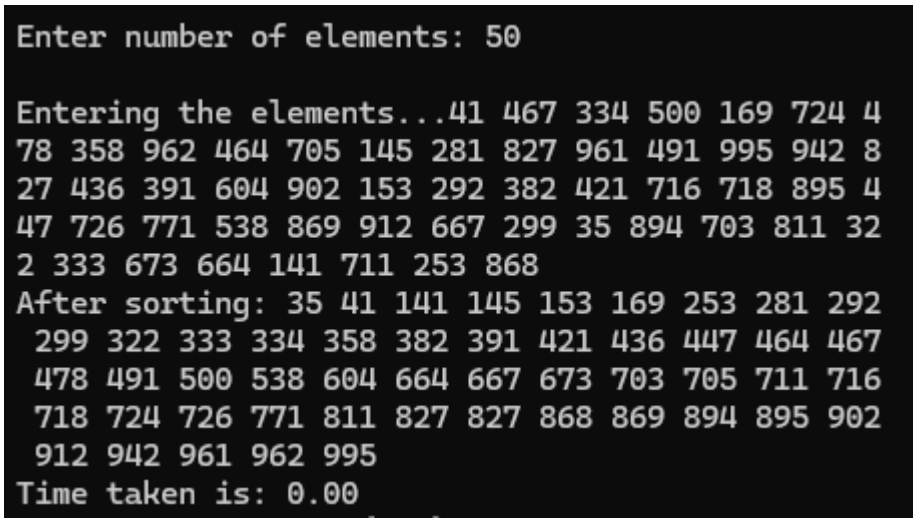
Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

SOURCE CODE

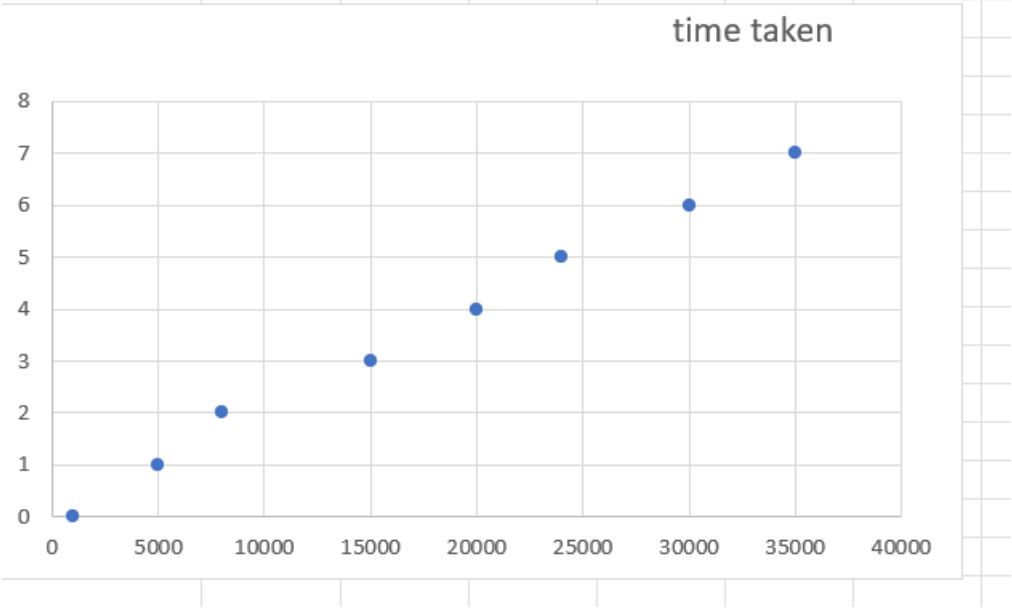
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int a[5000000],b[5000000];

void merge(int a[],int low,int mid,int high){

   int i=low,j=mid+1,k=low;

   while(i<=mid && j<=high){
      if(a[i]<a[j])
         b[k]=a[i++];
      else
         b[k]=a[j++];
      k++;
   }
   while(i<=mid){
      b[k++]=a[i++];
   }
   while(j<=high){
      b[k++]=a[j++];
   }
   for(int x=low;x<=high;x++)
      a[x]=b[x];
}


void merge_sort(int a[],int low,int high){
   if(low<high){
      int mid=(low+high)/2;
      merge_sort(a,low,mid);
      merge_sort(a,mid+1,high);
      merge(a,low,mid,high);
   }
}
```

```
int main(){
   int n;
   printf("\nEnter number of elements: ");
   scanf("%d",&n);
   printf("\nEntering the elements...");
   for(int i=0;i<n;i++){
      a[i]=rand()%1000;
      printf("%d ",a[i]);
   }
   clock_t start_time=clock();
   merge_sort(a,0,n-1);
   clock_t end_time=clock();
   printf("\nAfter sorting: ");
   for(int i=0;i<n;i++)
      printf("%d ",a[i]);
   float total_time=(float)(end_time-start_time)/CLOCKS_PER_SEC*1000;
   printf("\nTime taken is: %.2f",total_time);
   return 0;
}
```

OUTPUT



no. of elements	time taken
1000	0
5000	1
8000	2
15000	3
20000	4
24000	5
30000	6
35000	7



LAB PROGRAM 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

SOURCE CODE

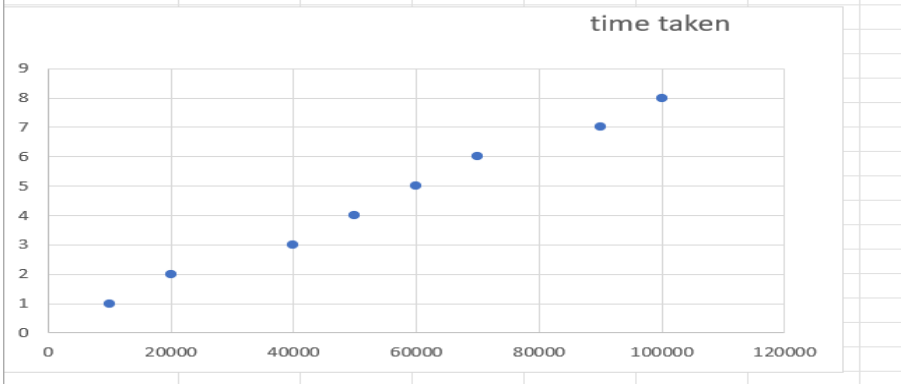
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void exchange(int *a, int *b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}

int partition(int low, int high, int a[]) {
   int pivot = a[low];
   int i = low + 1;
   int j = high;

   while (i <= j) {
      while (i <= high && a[i] <= pivot) i++;
      while (j >= low && a[j] > pivot) j--;
      if (i < j) exchange(&a[i], &a[j]);
   }
   exchange(&a[low], &a[j]);
   return j;
}

void quick_sort(int low, int high, int a[]) {
   if (low < high) {
      int j = partition(low, high, a);
      quick_sort(low, j - 1, a);
      quick_sort(j + 1, high, a);
   }
}

int main() {
   int n;
   printf("Enter number of elements: ");
   scanf("%d", &n);

   int a[n];
   printf("Unsorted elements:\n");
   for (int i = 0; i < n; i++) {
      a[i] = rand() % 1000;
      printf("%d ", a[i]);
   }
```

```
    clock_t start = clock();
    quick_sort(0, n - 1, a);
    clock_t end = clock();

    double time_taken = (double)(end - start) *1000/ CLOCKS_PER_SEC;

    printf("\nSorted elements:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nTime taken: %f seconds\n", time_taken);
    return 0;
}
```

OUTPUT

```
Enter number of elements: 50
Unsorted elements:
41 467 334 500 169 724 478 358 962 464 705 145 281 827 961 491 995 942 827 436
391 604 902 153 292 382 421 716 718 895 447 726 771 538 869 912 667 299 35 894
703 811 322 333 673 664 141 711 253 868
Sorted elements:
35 41 141 145 153 169 253 281 292 299 322 333 334 358 382 391 421 436 447 464 4
67 478 491 500 538 604 664 667 673 703 705 711 716 718 724 726 771 811 827 827
868 869 894 895 902 912 942 961 962 995
Time taken: 0.000000 seconds

Process returned 0 (0x0)   execution time : 10.102 s
Press any key to continue.
```

no. of elements	time taken
10000	1
20000	2
40000	3
50000	4
60000	5
70000	6
90000	7
100000	8
110000	9



LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

SOURCE CODE

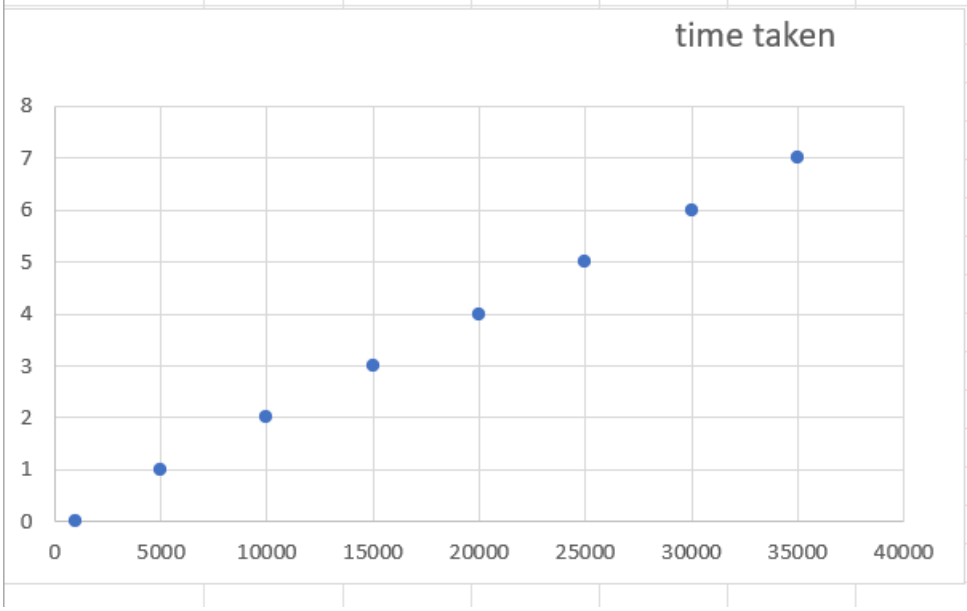
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void heapify(int a[], int n, int i) {
   int largest = i, l = 2*i + 1, r = 2*i + 2, temp;

   if (l < n && a[l] > a[largest]) largest = l;
   if (r < n && a[r] > a[largest]) largest = r;

   if (largest != i) {
      temp = a[i];
      a[i] = a[largest];
      a[largest] = temp;

      heapify(a, n, largest);
   }
}

void heapSort(int a[], int n) {
   int i, temp;

   for (i = n/2 - 1; i >= 0; i--)
      heapify(a, n, i);

   for (i = n-1; i > 0; i--) {
      temp = a[0];
      a[0] = a[i];
      a[i] = temp;

      heapify(a, i, 0);
   }
}

int main() {
   int n, i;

   printf("Enter number of inputs: ");
   scanf("%d", &n);

   int a[n];

   srand(time(0));
```

```
    // Generate random numbers
    for (i = 0; i < n; i++)
        a[i] = rand() % 1000;

    clock_t start, end;

    start = clock();

    heapSort(a, n);

    end = clock();

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nSorted elements:\n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n\nExecution Time = %f seconds\n", time_taken);

    return 0;
}
```

OUTPUT

```
Enter number of inputs: 30

Sorted elements:
80 81 104 162 182 194 229 246 249 254 299 402 451 461 472 527 579 591 597 674 678 692 717 735 770 798 897 923 924 992

Execution Time = 0.000000 seconds

Process returned 0 (0x0)   execution time : 6.423 s
Press any key to continue.
```

range	time taken
1000	0
5000	1
10000	2
15000	3
20000	4
25000	5
30000	6
35000	7



LAB PROGRAM 6:

Implement 0/1 Knapsack problem using dynamic programming.

SOURCE CODE

```
#include <stdio.h>
#include <time.h>

#define N 4
#define W 8

int max(int a, int b) {
   return (a > b) ? a : b;
}

int main() {
   int wt[N] = {3,4,6,5};
   int val[N] = {2,3,1,4};

   int dp[N+1][W+1];

   clock_t start = clock();

   for(int i = 0; i <= N; i++) {
      for(int w = 0; w <= W; w++) {
         if(i == 0 || w == 0)
            dp[i][w] = 0;
         else if(wt[i-1] <= w)
            dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]], dp[i-1][w]);
         else
            dp[i][w] = dp[i-1][w];
      }
   }

   clock_t end = clock();

   printf("Max value = %d\n", dp[N][W]);
   printf("Execution time = %lf sec\n", (double)(end - start)/CLOCKS_PER_SEC);

   printf("\nDP Matrix:\n");
   for(int i = 0; i <= N; i++) {
      for(int w = 0; w <= W; w++) {
         printf("%3d ", dp[i][w]);
      }
      printf("\n");
   }

   return 0;
}
```

OUTPUT

```
Max value = 6
Execution time = 0.000000 sec

DP Matrix:
  0   0   0   0   0   0   0   0   0
  0   0   0   2   2   2   2   2   2
  0   0   0   2   3   3   3   5   5
  0   0   0   2   3   3   3   5   5
  0   0   0   2   3   4   4   5   6
```

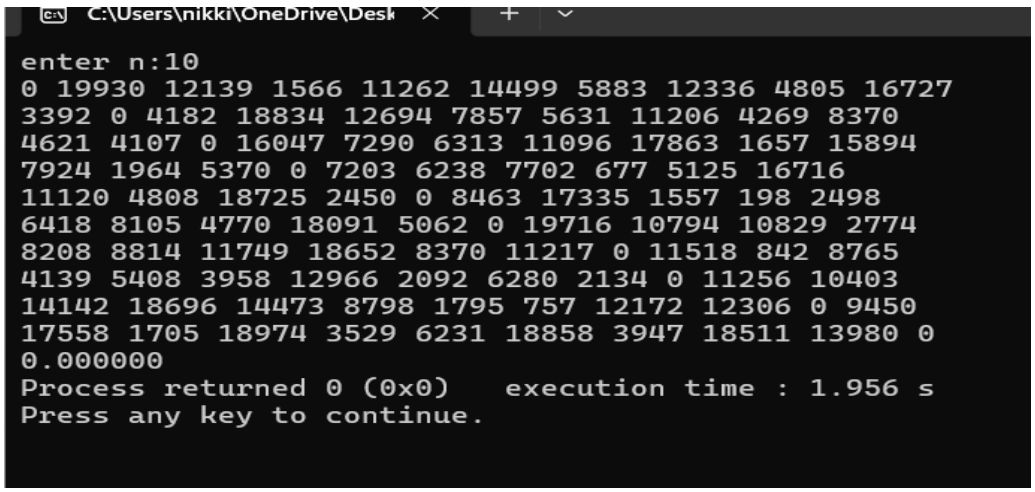
LAB PROGRAM 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

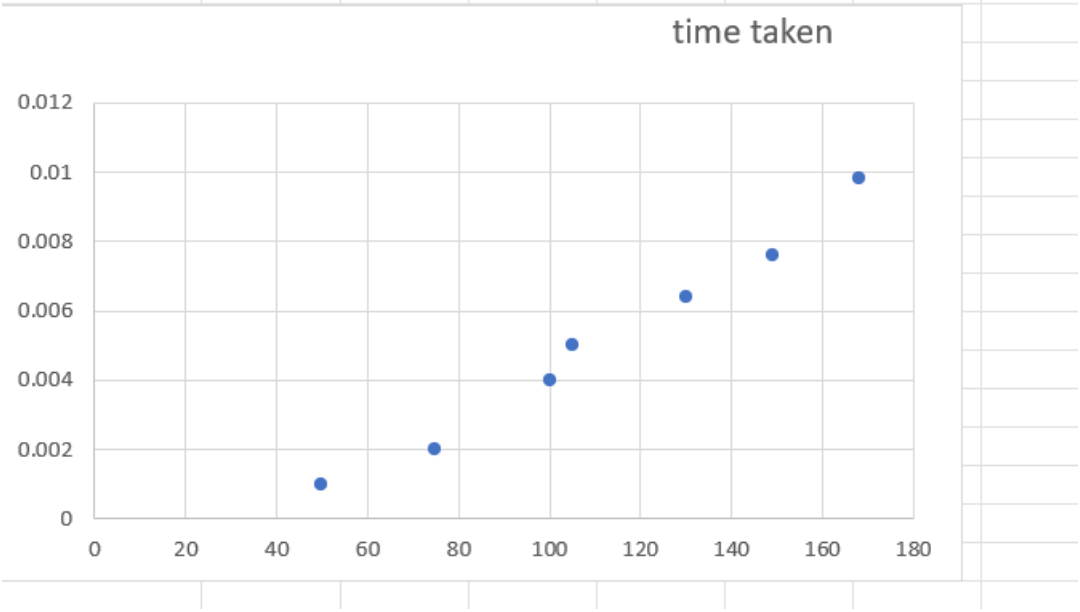
SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int d[100][100], i, j, k,n ;

main() {
   srand(time(0));
   printf ("enter n:");
   scanf("%d",&n);
   for (i = 0; i < n; i++){

      for (j = 0; j < n; j++){
         d[i][j] = i == j ? 0 : rand() % 20000 + 1;
         printf("%d ",d[i][j]);
      } printf("\n");
   }
   clock_t s = clock();

   for (k = 0; k < n; k++)
      for (i = 0; i < n; i++)
         for (j = 0; j < n; j++)
            if (d[i][k] + d[k][j] < d[i][j])
               d[i][j] = d[i][k] + d[k][j];

   printf("%f", (double)(clock() - s) / CLOCKS_PER_SEC);
}
```

OUTPUT



no. of elements	time taken
50	0.001
75	0.002
100	0.004
105	0.005
130	0.0064
149	0.0076
168	0.0098



LAB PROGRAM 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define INF 9999

int parent[100] = {0};

int find(int i){
   if(parent[i] == 0)
      return i;
   return parent[i] = find(parent[i]);
}

int uni(int i,int j){
   if(i != j){
      parent[j] = i;
      return 1;
   }
   return 0;
}

int main(){
   int n,c[100][100],i,j,choice;
   clock_t start,end;

   printf("Enter number of vertices: ");
   scanf("%d",&n);

   srand(time(0));


   for(i=0;i<n;i++){
      for(j=0;j<n;j++){
         if(i == j)
            c[i][j] = INF;
         else{
            int w = rand() % 100 + 1;
            c[i][j] = c[j][i] = w;
         }
      }
   }
```

```
printf("Enter 1 for Prim, 2 for Kruskal: ");
scanf("%d",&choice);

start = clock();

if(choice==1){
   int vis[100]={0},min,u,v,a,b,ne=1,total=0;

   vis[0]=1;

   while(ne < n){
      min = INF;

      for(i=0;i<n;i++){
         if(vis[i]){
            for(j=0;j<n;j++){
               if(!vis[j] && c[i][j] < min){
                  min = c[i][j];
                  a = i;
                  b = j;
               }
            }
         }
      }

      total += min;
      vis[b] = 1;
      ne++;
   }

   printf("Cost = %d\n", total);
}

else{
   int min,u,v,a,b,ne=1,total=0;

   while(ne < n){
      min = INF;

      for(i=0;i<n;i++){
         for(j=0;j<n;j++){
            if(i != j && c[i][j] < min){
               min = c[i][j];
               a = u = i;
               b = v = j;
            }
         }
      }

      u = find(u);
```

```
            v = find(v);

            if(uni(u,v)){
               total += min;
               ne++;
            }

            c[a][b] = c[b][a] = INF;
         }

         printf("Cost = %d\n", total);
      }
      printf("\nGenerated Cost Matrix:\n");

   for(i = 0; i < n; i++){
      for(j = 0; j < n; j++){
         if(c[i][j] == INF)
            printf("INF ");
         else
            printf("%3d ", c[i][j]);
      }
      printf("\n");
   }

      end = clock();

      double time_taken = (double)(end - start) / CLOCKS_PER_SEC * 1000;
      printf("Time = %.3f ms\n", time_taken);

      return 0;
   }
```

OUTPUT

```
Enter number of vertices: 10
Enter 1 for Prim, 2 for Kruskal: 1
Cost = 93

Generated Cost Matrix:
INF   2  28  59   2  15  80  85  48  15
  2 INF  79   9  27  58  45  87 100  29
 28  79 INF  72  20  41  11  65  36  90
 59   9  72 INF  52  40  69  58  33  72
  2  27  20  52 INF  14   5  22  62  16
 15  58  41  40  14 INF  95 100  45  98
 80  45  11  69   5  95 INF  12  48  22
 85  87  65  58  22 100  12 INF  73   5
 48 100  36  33  62  45  48  73 INF  43
 15  29  90  72  16  98  22   5  43 INF
Time = 10.000 ms
```

```
Enter number of vertices: 10
Enter 1 for Prim, 2 for Kruskal: 2
Cost = 85

Generated Cost Matrix:
INF  42 INF  26  98  35  61  92 100  74
 42 INF  30  67  89 INF  51  82  37  59
INF  30 INF  96  65  67  47  40 INF  48
 26  67  96 INF  26  80  32 INF INF INF
 98  89  65  26 INF INF  38  50 INF  77
 35 INF  67  80 INF INF INF  99 INF  23
 61  51  47  32  38 INF INF  44  79  87
 92  82  40 INF  50  99  44 INF  66  66
100  37 INF INF INF INF  79  66 INF  26
 74  59  48 INF  77  23  87  66  26 INF
Time = 14.000 ms
```

LAB PROGRAM 9:

Implement Fractional Knapsack using Greedy technique.

SOURCE CODE

```
#include <stdio.h>

struct Item {
   int id;
   float profit;
   float weight;
   float ratio;
};

void heapify(struct Item arr[], int n, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   if (left < n && arr[left].ratio > arr[largest].ratio)
      largest = left;

   if (right < n && arr[right].ratio > arr[largest].ratio)
      largest = right;

   if (largest != i) {
      struct Item temp = arr[i];
      arr[i] = arr[largest];
      arr[largest] = temp;
      heapify(arr, n, largest);
   }
}


void heapSort(struct Item arr[], int n) {

   for (int i = n / 2 - 1; i >= 0; i--)
      heapify(arr, n, i);


   for (int i = n - 1; i > 0; i--) {
      struct Item temp = arr[0];
      arr[0] = arr[i];
      arr[i] = temp;
      heapify(arr, i, 0);
   }
}
```

```
void fractionalKnapsack(int n, struct Item items[], float capacity) {

    for (int i = 0; i < n; i++) {
        items[i].ratio = items[i].profit / items[i].weight;
    }


    heapSort(items, n);

    float totalProfit = 0.0;
    printf("\n--- Selection (Greedy via Heap Sort) ---\n");

    for (int i = n - 1; i >= 0; i--) {
        if (capacity <= 0) break;

        if (items[i].weight <= capacity) {
            capacity -= items[i].weight;
            totalProfit += items[i].profit;
            printf("Added Item %d (100%%). Remaining Capacity: %.2f\n", items[i].id,
capacity);
        } else {
            float fraction = capacity / items[i].weight;
            totalProfit += items[i].profit * fraction;
            printf("Added Item %d (%.0f%%). Remaining Capacity: 0.00\n", items[i].id, fraction
* 100);
            capacity = 0;
        }
    }
    printf("\nMaximum Profit: %.2f\n", totalProfit);
}

int main() {
    int n;
    float capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item items[n];
    for (int i = 0; i < n; i++) {
        items[i].id = i + 1;
        printf("Enter profit and weight for item %d: ", i + 1);
        scanf("%f %f", &items[i].profit, &items[i].weight);
    }

    printf("Enter knapsack capacity: ");
    scanf("%f", &capacity);

    fractionalKnapsack(n, items, capacity);
```

```
    return 0;
}
```

OUTPUT

```
Enter number of items: 3
Enter profit and weight for item 1: 25 18
Enter profit and weight for item 2: 24 15
Enter profit and weight for item 3: 15 10
Enter knapsack capacity: 20

--- Selection (Greedy via Heap Sort) ---
Added Item 2 (100%). Remaining Capacity: 5.00
Added Item 3 (50%). Remaining Capacity: 0.00

Maximum Profit: 31.50

Process returned 0 (0x0)   execution time : 17.751 s
Press any key to continue.
```

LAB PROGRAM 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

SOURCE CODE

```
#include <stdio.h>
#include <limits.h>

#define V 5
int minDistance(int dist[], int visited[]) {
   int min = INT_MAX, min_index;

   for (int v = 0; v < V; v++) {
      if (!visited[v] && dist[v] <= min) {
         min = dist[v];
         min_index = v;
      }
   }
   return min_index;
}

void dijkstra(int graph[V][V], int src) {
   int dist[V];
   int visited[V];

   for (int i = 0; i < V; i++) {
      dist[i] = INT_MAX;
      visited[i] = 0;
   }

   dist[src] = 0;

   for (int count = 0; count < V - 1; count++) {
      int u = minDistance(dist, visited);
      visited[u] = 1;

      for (int v = 0; v < V; v++) {
         if (!visited[v] && graph[u][v] &&
            dist[u] != INT_MAX &&
            dist[u] + graph[u][v] < dist[v]) {

            dist[v] = dist[u] + graph[u][v];
         }
      }
   }

   printf("Vertex\tDistance from Source\n");
   for (int i = 0; i < V; i++) {
      printf("%d\t%d\n", i, dist[i]);
```

```
    }
}

int main() {
    int graph[V][V] = {
        {0, 10, 0, 30, 100},
        {10, 0, 50, 0, 0},
        {0, 50, 0, 20, 10},
        {30, 0, 20, 0, 60},
        {100, 0, 10, 60, 0}
    };

    int source = 0;
    dijkstra(graph, source);

    return 0;
}
```

OUTPUT

```
Vertex  Distance from Source
0       0
1       10
2       50
3       30
4       60
```

LAB PROGRAM 11:

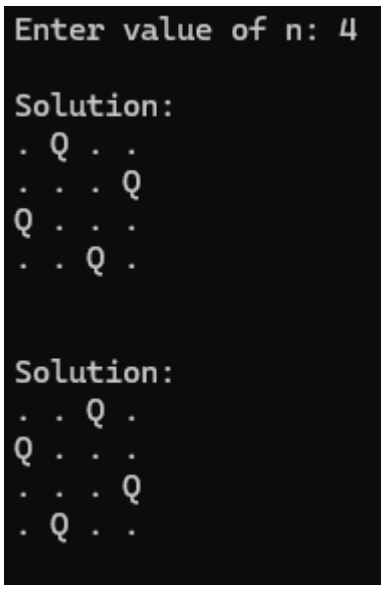
Implement “N-Queens Problem” using Backtracking.

SOURCE CODE

```
#include <stdio.h>

#define N 4

int board[N][N];

int isSafe(int row, int col)
{
   int i, j;
   for(i = 0; i < col; i++)
      if(board[row][i])
         return 0;

   for(i = row, j = col; i >= 0 && j >= 0; i--, j--)
      if(board[i][j])
         return 0;

   for(i = row, j = col; j >= 0 && i < N; i++, j--)
      if(board[i][j])
         return 0;

   return 1;
}

int solveNQueens(int col)
{
   if(col >= N)
      return 1;

   for(int i = 0; i < N; i++)
   {
      if(isSafe(i, col))
      {
         board[i][col] = 1;

         if(solveNQueens(col + 1))
            return 1;

         board[i][col] = 0; // Backtrack
      }
   }

   return 0;
}
```

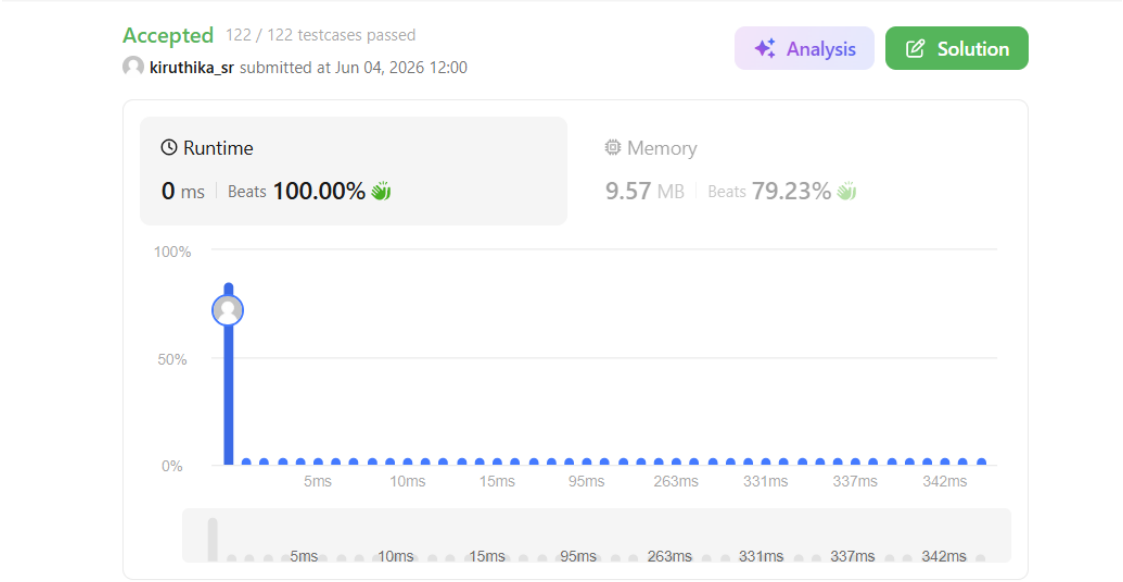
```
void printSolution()
{
   for(int i = 0; i < N; i++)
   {
      for(int j = 0; j < N; j++)
         printf("%d ", board[i][j]);

      printf("\n");
   }
}

int main()
{
   if(solveNQueens(0))
   {
      printf("Solution Found:\n");
      printSolution();
   }
   else
   {
      printf("No Solution Exists\n");
   }

   return 0;
}
```

OUTPUT



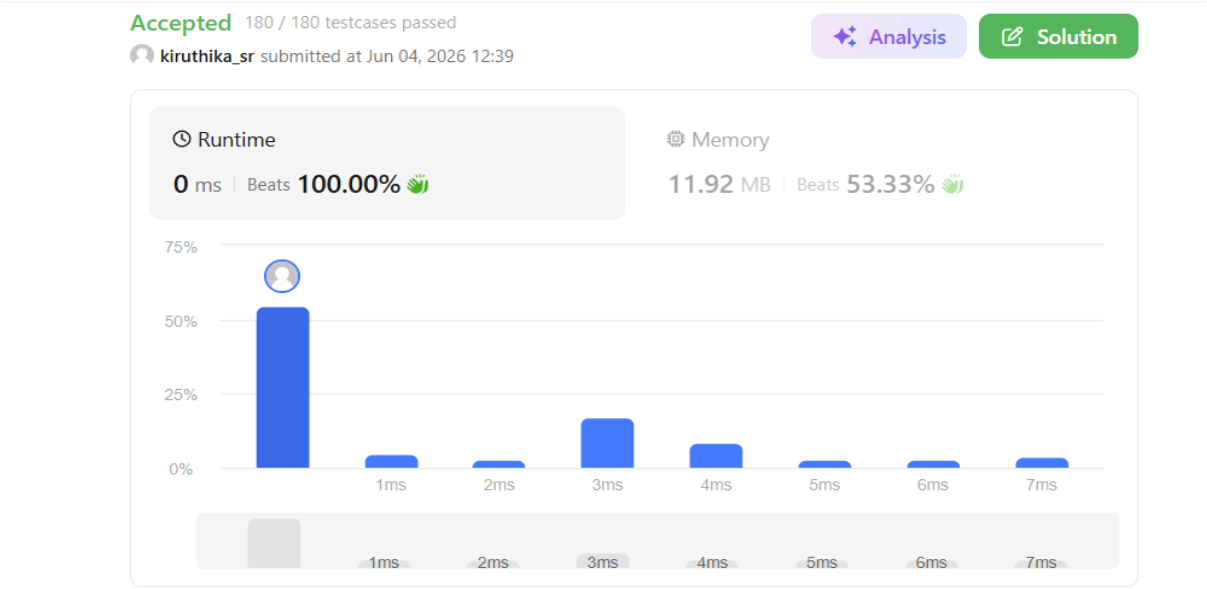
LEETCODE 268: MISSING NUMBER

```
#include <stdio.h>

int missingNumber(int* nums, int numsSize) {
    int expectedSum = numsSize * (numsSize + 1) / 2;
    int actualSum = 0;

    for (int i = 0; i < numsSize; i++) {
        actualSum += nums[i];
    }

    return expectedSum - actualSum;
}
```



LEETCODE 1636: SORT ARRAY BY INCREASING FREQUENCY

```
#include <stdlib.h>

int freq[201];

int compare(const void *a, const void *b) {
    int x = *(int *)a;
    int y = *(int *)b;

    if (freq[x + 100] == freq[y + 100]) {
        return y - x;  // decreasing order for same frequency
    }

    return freq[x + 100] - freq[y + 100];  // increasing frequency
}

int* frequencySort(int* nums, int numsSize, int* returnSize) {
    for (int i = 0; i < 201; i++) {
        freq[i] = 0;
    }

    for (int i = 0; i < numsSize; i++) {
        freq[nums[i] + 100]++;
    }

    qsort(nums, numsSize, sizeof(int), compare);

    *returnSize = numsSize;
    return nums;
```



Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =

```
[1,1,2,2,2,3]
```

Output

```
[3,1,1,2,2,2]
```

Expected

```
[3,1,1,2,2,2]
```

LEETCODE 148: SORT LIST

```
struct ListNode* merge(struct ListNode* left, struct ListNode* right) {

    struct ListNode dummy;
    struct ListNode* tail = &dummy;
    dummy.next = NULL;

    while (left && right) {
        if (left->val <= right->val) {
            tail->next = left;
            left = left->next;
        } else {
            tail->next = right;
            right = right->next;
        }
        tail = tail->next;
    }

    if (left)
        tail->next = left;
    else
        tail->next = right;

    return dummy.next;
}

struct ListNode* sortList(struct ListNode* head) {

    if (head == NULL || head->next == NULL)
        return head;

    struct ListNode* slow = head;
    struct ListNode* fast = head->next;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    struct ListNode* mid = slow->next;
    slow->next = NULL;

    struct ListNode* left = sortList(head);
    struct ListNode* right = sortList(mid);

    return merge(left, right);
}
```

Accepted 30 / 30 testcases passed

kiruthika_sr submitted at May 24, 2026 20:32

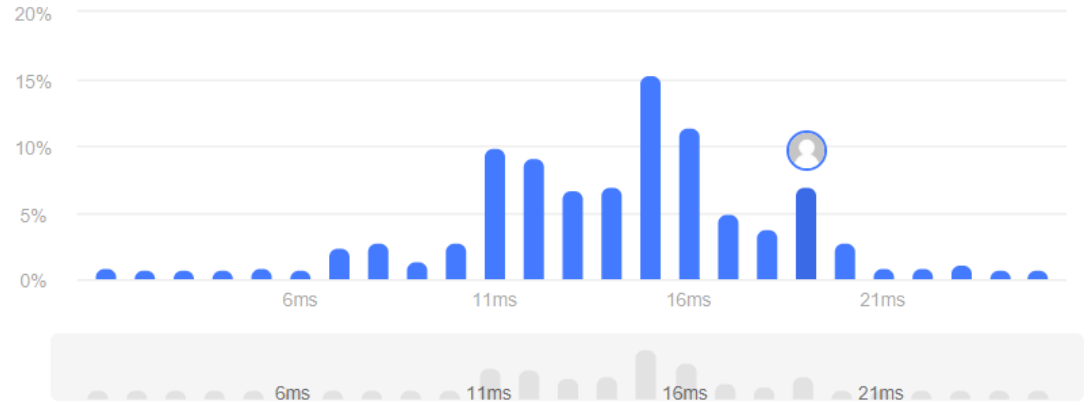
Analysis | Solution

Runtime

19 ms | Beats 18.72%

Memory

25.46 MB | Beats 73.59%



Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

head =
[4,2,1,3]

Output

[1,2,3,4]

Expected

[1,2,3,4]

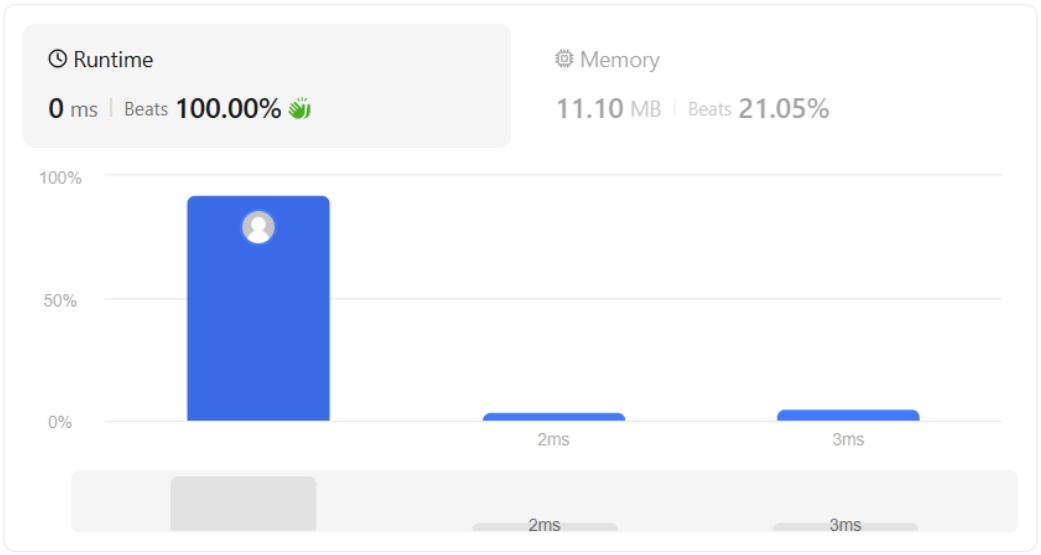
LEETCODE 969: PANKCAKE SORTING

```
void reverse(int* arr, int k) {
    int left = 0;
    int right = k - 1;

    while (left < right) {
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;

        left++;
        right--;
    }
}

int* pancakeSort(int* arr, int arrSize, int* returnSize) {

    int* result = (int*)malloc(sizeof(int) * (2 * arrSize));
    int idx = 0;

    for (int currSize = arrSize; currSize > 1; currSize--) {

        int maxIndex = 0;

        for (int i = 1; i < currSize; i++) {
            if (arr[i] > arr[maxIndex]) {
                maxIndex = i;
            }
        }

        if (maxIndex == currSize - 1)
            continue;

        if (maxIndex != 0) {
            reverse(arr, maxIndex + 1);
            result[idx++] = maxIndex + 1;
        }

        reverse(arr, currSize);
        result[idx++] = currSize;
    }

    *returnSize = idx;
    return result;
}
```

Accepted 215 / 215 testcases passed

kiruthika_sr submitted at May 24, 2026 20:34

Analysis Solution



Accepted Runtime: 0 ms

Case 1 Case 2

Input

arr =

```
[3,2,4,1]
```

Output

```
[3,4,2,3,2]
```

Expected

```
[4,2,4,3]
```

LEETCODE 496: NEXT GREATER ELEMENT I

```
int* nextGreaterElement(int* nums1, int nums1Size,
                        int* nums2, int nums2Size,
                        int* returnSize) {

    int map[10001];
    int stack[1000];
    int top = -1;

    for (int i = 0; i <= 10000; i++) {
        map[i] = -1;
    }

    for (int i = 0; i < nums2Size; i++) {

        while (top >= 0 && nums2[i] > stack[top]) {
            map[stack[top]] = nums2[i];
            top--;
        }

        stack[++top] = nums2[i];
    }

    int* ans = (int*)malloc(sizeof(int) * nums1Size);

    for (int i = 0; i < nums1Size; i++) {
        ans[i] = map[nums1[i]];
    }

    *returnSize = nums1Size;
    return ans;
}
```

Accepted 17 / 17 testcases passed

kiruthika_sr submitted at May 24, 2026 20:36

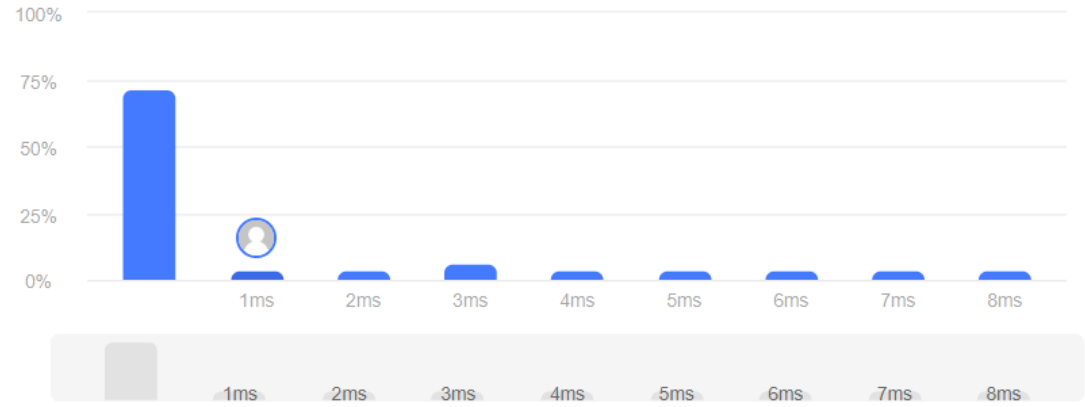
Analysis | Solution

Runtime

1 ms | Beats 28.54%

Memory

12.56 MB | Beats 6.86%



Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums1 =

```
[4,1,2]
```

nums2 =

```
[1,3,4,2]
```

Output

```
[-1,3,-1]
```

Expected

```
[-1,3,-1]
```

LEETCODE 787: CHEAPEST FLIGHTS WITHIN K STOPS

```
#include <limits.h>
#include <string.h>

int findCheapestPrice(int n, int** flights, int flightsSize,
                      int* flightsColSize, int src, int dst, int k) {

    int cost[101];

    for (int i = 0; i < n; i++)
        cost[i] = INT_MAX;

    cost[src] = 0;

    for (int step = 0; step <= k; step++) {
        int temp[101];

        memcpy(temp, cost, sizeof(cost));

        for (int i = 0; i < flightsSize; i++) {
            int u = flights[i][0];
            int v = flights[i][1];
            int price = flights[i][2];

            if (cost[u] != INT_MAX &&
                cost[u] + price < temp[v]) {
                temp[v] = cost[u] + price;
            }
        }

        memcpy(cost, temp, sizeof(cost));
    }

    return (cost[dst] == INT_MAX) ? -1 : cost[dst];
```

Accepted 59 / 59 testcases passed

kiruthikazz submitted at Jun 01, 2026 10:29

Analysis Solution

Runtime

0 ms | Beats 100.00%

Memory

11.58 MB | Beats 25.83%

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

n =
4

flights =
[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]

src =
0

dst =
3

k =
1

Output

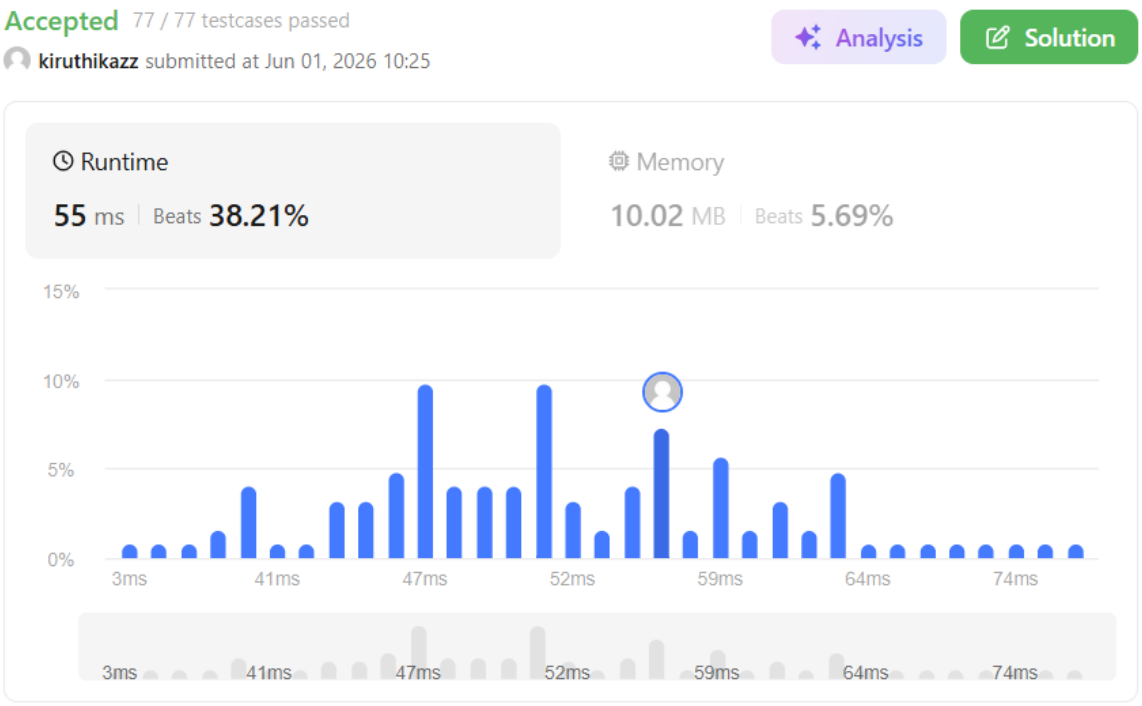
700

Expected

700

LEETCODE 474: ONES AND ZEROES

```
int max(int a, int b) {
   return (a > b) ? a : b;
}

int findMaxForm(char **strs, int strsSize, int m, int n) {
   int dp[101][101] = {0};

   for (int k = 0; k < strsSize; k++) {
      int zeros = 0, ones = 0;

      for (int i = 0; strs[k][i] != '\0'; i++) {
         if (strs[k][i] == '0')
            zeros++;
         else
            ones++;
      }

      for (int i = m; i >= zeros; i--) {
         for (int j = n; j >= ones; j--) {
            dp[i][j] = max(dp[i][j],
                           dp[i - zeros][j - ones] + 1);
         }
      }
   }

   return dp[m][n];
}
```



Accepted Runtime: 0 ms

Case 1 Case 2

Input

strs =

```
["10","0001","111001","1","0"]
```

m =

5

n =

3

Output

4

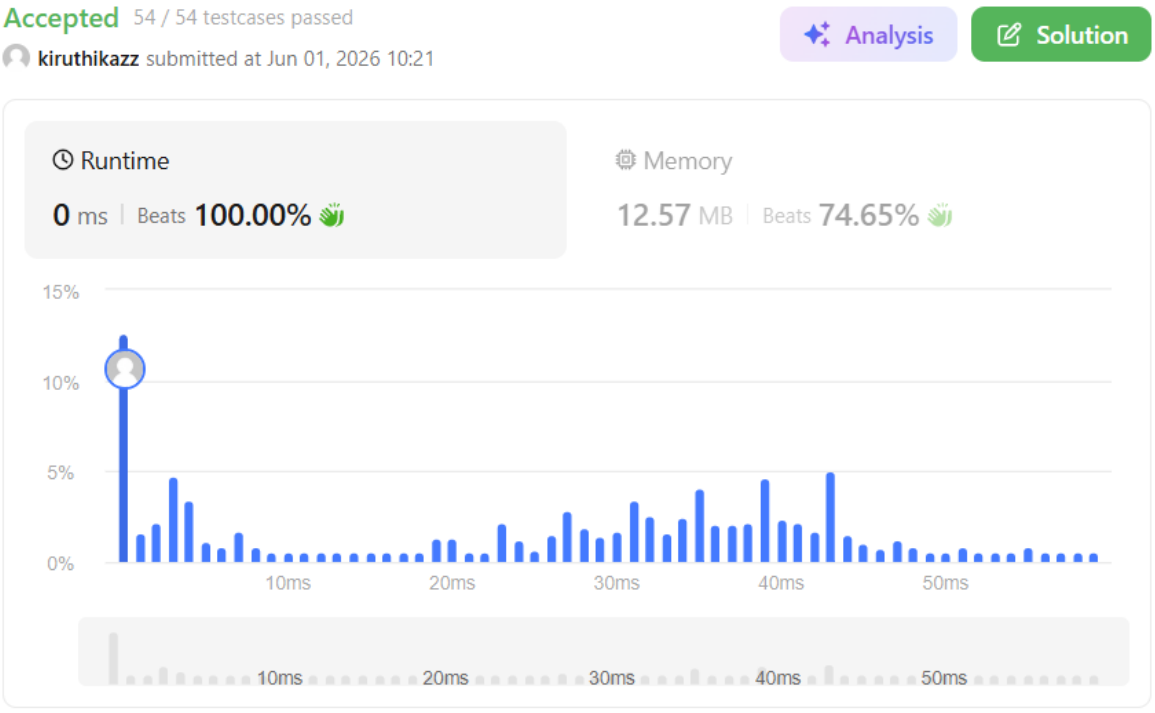
Expected

4

LEETCODE 207: COURSE SCHEDULE

```
#include <stdlib.h>
#include <stdbool.h>

bool canFinish(int numCourses, int** prerequisites,
               int prerequisitesSize,
               int* prerequisitesColSize) {

    int **adj = (int **)malloc(numCourses * sizeof(int *));
    int *adjSize = (int *)calloc(numCourses, sizeof(int));

    for (int i = 0; i < prerequisitesSize; i++) {
        int prereq = prerequisites[i][1];
        adjSize[prereq]++;
    }

    for (int i = 0; i < numCourses; i++) {
        adj[i] = (int *)malloc(adjSize[i] * sizeof(int));
        adjSize[i] = 0;
    }

    int *indegree = (int *)calloc(numCourses, sizeof(int));

    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];

        adj[prereq][adjSize[prereq]++] = course;
        indegree[course]++;
    }

    int *queue = (int *)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;

    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }

    int completed = 0;

    while (front < rear) {
        int curr = queue[front++];
        completed++;

        for (int i = 0; i < adjSize[curr]; i++) {
            int next = adj[curr][i];
```

```
            indegree[next]--;

            if (indegree[next] == 0) {
                queue[rear++] = next;
            }
        }
    }
    for (int i = 0; i < numCourses; i++) {
        free(adj[i]);
    }
    free(adj);
    free(adjSize);
    free(indegree);
    free(queue);

    return completed == numCourses;
}
```



Accepted Runtime: 0 ms

Case 1 Case 2

Input

numCourses =

2

prerequisites =

[[1,0]]

Output

true

Expected

true